

Subgraph Algorithmus und optimale Polygon-Tessellierung in der Echtzeit-Computergrafik

Stephan Epp

16. April 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Problemstellung	3
1.2	Motivation	3
2	Grundlagen	3
2.1	Definitionen	3
2.2	Grundidee	3
2.2.1	Eindeutige Signaturen	4
2.2.2	Zyklische Rotation statt Permutation	4
3	Algorithmus	4
3.1	Algorithmus-Beschreibung	4
3.2	Vergleich der Signatur-Sequenzen	5
3.3	Implementierung	5
4	Analyse	5
4.1	Laufzeit	5
4.2	Optimalität der Laufzeit	6
4.3	Korrektheit	6
5	Optimale Polygon-Tessellierung	6
5.1	Polygone als Graphen: formale Einbettung	6
5.2	Zyklische Symmetrie von Polygon-Graphen	7
5.3	Das Effizienzmaß für Polygon-Primitive	8
5.4	Universeller Zerlegungssatz	10
5.5	Subgraph-Relation zwischen Dreiecks- und Pentagon-Graph	11
5.6	Der Subgraph Algorithmus als Optimierungswerkzeug	12
5.7	Optimale Dreiecksdichte: Effizienz-Realitäts-Trade-off	12
5.8	Die Sonderrolle des Pentagons	13
5.9	Gesamtsystem: Subgraph-basierte Szenen-Optimierung	14

6	Zusammenfassung	15
6.1	Anwendungen	15
6.2	Ausblick	16

1 Einführung

Der Subgraph Algorithmus ist ein effizienter Algorithmus zum Vergleichen zweier Graphen G und G' mittels Adjazenzmatrizen und Signatur-Arrays. Der Algorithmus bestimmt durch zyklische Rotation, ob ein Graph als Subgraph in einem anderen enthalten ist.

1.1 Problemstellung

Gegeben sind zwei Graphen G und G' mit jeweils n Knoten, repräsentiert durch Adjazenzmatrizen A und B .

Ziel: Bestimme, ob Graph G in Graph G' enthalten ist, unter Berücksichtigung aller möglichen Knotenzuordnungen durch zyklische Rotation.

Ergebnis:

- Wenn $B \supseteq A$: Verwerfe A , behalte B (G' hat mehr Informationen)
- Wenn $A \supseteq B$: Behalte A , verwerfe B (G hat mehr Informationen)
- Wenn beide identisch sind: Beliebige behalten
- Wenn keiner den anderen enthält: Beide behalten

1.2 Motivation

Der Subgraph Algorithmus eignet sich besonders für die Verifikation von unterschiedlichen Programmen durch die Analyse der Abstract Syntax Trees (AST). Die Methode ermöglicht es, strukturelle Ähnlichkeiten zwischen Programmrepräsentationen effizient zu erkennen.

2 Grundlagen

2.1 Definitionen

Definition 2.1 (Graph). Ein Graph $G = (V, E)$ besteht aus einer Menge von Knoten $V = \{v_1, v_2, \dots, v_n\}$ und einer Menge von Kanten $E \subseteq V \times V$.

Definition 2.2 (Adjazenzmatrix). Die Adjazenzmatrix A eines Graphen $G = (V, E)$ mit n Knoten ist eine $n \times n$ Matrix mit:

$$A_{ij} = \begin{cases} 1 & \text{falls } (v_i, v_j) \in E \\ 0 & \text{sonst} \end{cases}$$

Definition 2.3 (Subgraph). Ein Graph $G = (V, E)$ ist ein Subgraph von $G' = (V', E')$, geschrieben $G \subseteq G'$, wenn $V \subseteq V'$ und $E \subseteq E'$.

Definition 2.4 (Zyklische Rotation). Eine zyklische Rotation einer Sequenz $[a_0, a_1, \dots, a_{n-1}]$ um k Positionen nach rechts ist definiert als:

$$\text{rotate}_k([a_0, a_1, \dots, a_{n-1}]) = [a_k, a_{k+1}, \dots, a_{n-1}, a_0, \dots, a_{k-1}]$$

2.2 Grundidee

Der Algorithmus basiert auf zwei zentralen Konzepten:

2.2.1 Eindeutige Signaturen

Für jede Spalte j der Adjazenzmatrix wird eine eindeutige Signatur berechnet:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$$

Beispiel 2.1. Für $n = 4$ und Spalte $j = 0$ mit Vektor $[1, 0, 1, 0]^T$:

$$\sigma_0 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 0 \cdot 2^4 = 1 + 4 = 5$$

Für Spalte $j = 1$ mit gleichem Vektor $[1, 0, 1, 0]^T$:

$$\sigma_1 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 = 1 + 4 + 16 = 21$$

Lemma 2.1 (Eindeutigkeit der Signaturen). Die Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv.

Beweis. Die Zeilenkomponente $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ kodiert jede Binärkombination eindeutig als Dezimalzahl. Die Spaltengewichtung $j \cdot 2^n$ unterscheidet gleiche Muster an verschiedenen Positionen, da $j \cdot 2^n$ für $j \in \{0, \dots, n-1\}$ stets disjunkte Intervalle $[j \cdot 2^n, (j+1) \cdot 2^n)$ erzeugt. $\square$

2.2.2 Zyklische Rotation statt Permutation

Statt alle $n!$ Permutationen zu prüfen, werden nur n zyklische Rotationen betrachtet. Dies erhält die sequentielle Ordnung und reduziert die Komplexität drastisch.

Beispiel 2.2. Für $n = 4$ Spalten:

Original:	$[\sigma_0, \sigma_1, \sigma_2, \sigma_3]$
Rotation 1:	$[\sigma_1, \sigma_2, \sigma_3, \sigma_0]$
Rotation 2:	$[\sigma_2, \sigma_3, \sigma_0, \sigma_1]$
Rotation 3:	$[\sigma_3, \sigma_0, \sigma_1, \sigma_2]$

Dies sind nur 4 Rotationen, nicht $4! = 24$ Permutationen.

3 Algorithmus

3.1 Algorithmus-Beschreibung

Der Algorithmus arbeitet in drei Hauptschritten:

Schritt 1: Signatur-Berechnung für Graph G

Berechne für jede Spalte j der Adjazenzmatrix A die Signatur:

$$\sigma_j^A = \sum_{i=0}^{n_A-1} A_{ij} \cdot 2^i + j \cdot 2^{n_A}$$

Schritt 2: Signatur-Berechnung für Graph G'

Berechne analog für Matrix B :

$$\sigma_j^B = \sum_{i=0}^{n_B-1} B_{ij} \cdot 2^i + j \cdot 2^{n_B}$$

Schritt 3: Rotation-Match

Für jede der n_B zyklischen Rotationen von B :

1. Rotiere Spalten zyklisch nach rechts
2. Extrahiere Zeilenkomponenten (ohne Spaltengewichtung)
3. Vergleiche Signatur-Sequenzen mit Longest Common Subsequence (LCS)
4. Falls $\text{LCS} \geq 2$: Subgraph-Beziehung gefunden

3.2 Vergleich der Signatur-Sequenzen

Satz 3.1 (Subgraph-Kriterium). Eine Subgraph-Beziehung existiert genau dann, wenn die längste gemeinsame Teilsequenz der Signatur-Arrays mindestens die Länge 2 hat.

Beweis. Eine Subgraph-Beziehung kann nur dann existieren, wenn mindestens zwei Knoten durch eine Kante miteinander verbunden sind. Dies entspricht einer gemeinsamen Teilsequenz der Länge mindestens 2 in den Signatur-Arrays. $\square$

3.3 Implementierung

Listing 1: Signatur-Berechnung

```
1 def _compute_column_signature(self, matrix):
2     n = matrix.shape[0]
3     signatures = []
4     for col in range(n):
5         column_vector = matrix[:, col]
6         row_signature = sum(2**i for i in range(n)
7                             if column_vector[i] == 1)
8         col_weight = col * (2**n)
9         signatures.append(row_signature + col_weight)
10    return signatures
```

4 Analyse

4.1 Laufzeit

Satz 4.1 (Laufzeit). Der Subgraph Algorithmus arbeitet mit einer Laufzeit von $O(n^3)$.

Beweis. Signatur-Berechnung: $O(n^2)$. Rotationsschleife mit n Iterationen, je $O(n^2)$ für LCS: $O(n^3)$. Gesamt: $O(n^3)$. $\square$

4.2 Optimalität der Laufzeit

Satz 4.2 (Optimalität). Unter SETH ist der Subgraph Algorithmus asymptotisch optimal: kein Algorithmus löst das zyklische Subgraph-Problem in $O(n^{3-\varepsilon})$ Zeit für $\varepsilon > 0$.

4.3 Korrektheit

Satz 4.3 (Korrektheit). Der Subgraph Algorithmus bestimmt korrekt, ob eine Subgraph-Beziehung zwischen zwei Graphen existiert.

Beweis. Die Rotation der Spalten entspricht dem Drehen des Graphen; die Struktur bleibt erhalten. Die Injektivität der Signatur-Funktion garantiert keine Fehlerkennungen. $\square$

5 Optimale Polygon-Tessellierung

Dieses Kapitel führt eine neue Anwendung des Subgraph Algorithmus ein: die graphentheoretische Analyse und Optimierung von Polygon-Primitiven in der Echtzeit-Computergrafik. Wir zeigen, dass der Subgraph Algorithmus nicht nur Graphen vergleicht, sondern als *strukturelles Klassifikations- und Zerlegungswerkzeug* für Polygonnetze dient – mit dem Ergebnis, dass das Dreieck das kanonisch optimale Primitiv und das Pentagon das natürlich optimale Zyklusprimitiv ist.

5.1 Polygone als Graphen: formale Einbettung

Definition 5.1 (Polygon-Graph). Ein n -Polygon-Graph $\Pi_n = (V_n, E_n)$ ist ein einfacher, ungerichteter Zykelgraph mit

$$V_n = \{v_0, v_1, \dots, v_{n-1}\}, \quad E_n = \{\{v_i, v_{(i+1) \bmod n}\} \mid i = 0, \dots, n-1\}.$$

Die zugehörige Adjazenzmatrix $A^{(n)} \in \{0, 1\}^{n \times n}$ lautet:

$$A_{ij}^{(n)} = \begin{cases} 1 & \text{falls } j \equiv i + 1 \pmod{n} \text{ oder } i \equiv j + 1 \pmod{n}, \\ 0 & \text{sonst.} \end{cases}$$

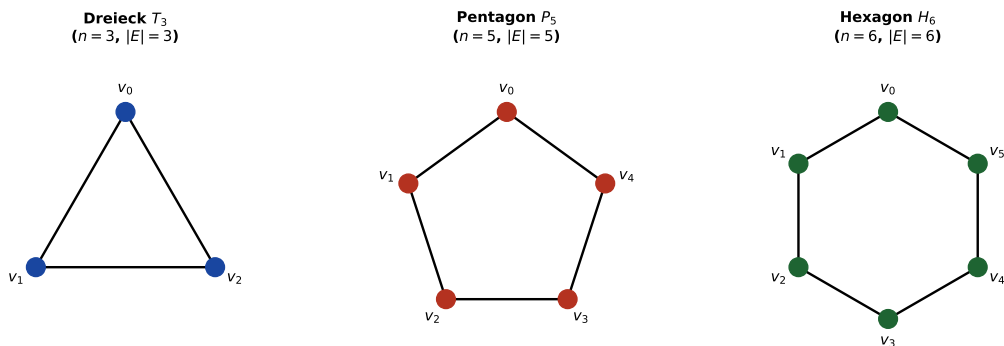


Abbildung 1: Polygon-Graphen Π_3 (Dreieck), Π_5 (Pentagon) und Π_6 (Hexagon) als ungerichtete Zyklusgraphen. Jeder Knoten v_i repräsentiert einen geometrischen Vertex.

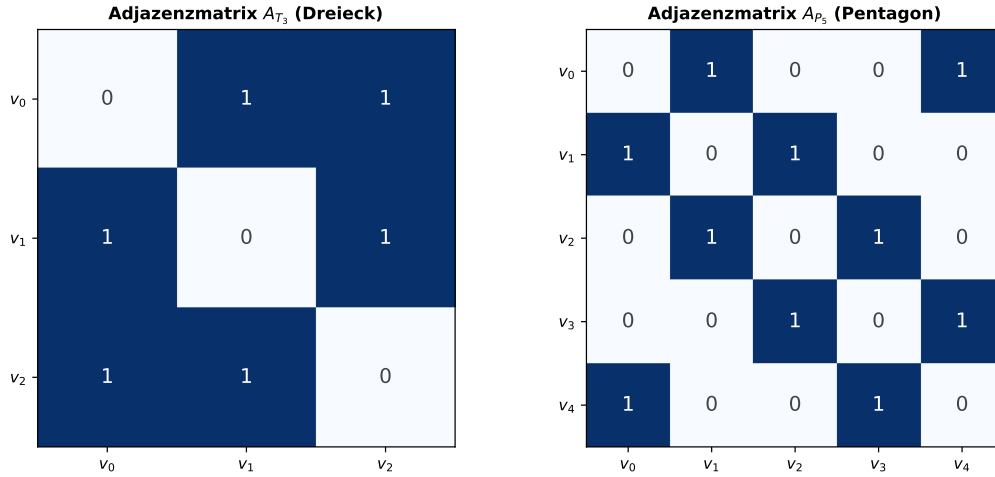


Abbildung 2: Adjazenzmatrizen $A^{(3)}$ des Dreiecks (links) und $A^{(5)}$ des Pentagons (rechts). Die Zyklusstruktur ist als Nebendiagonalenmuster erkennbar.

Lemma 5.1 (Signatur des Polygon-Graphen). Die Signaturfolge $\sigma^{(n)} = [\sigma_0^{(n)}, \dots, \sigma_{n-1}^{(n)}]$ des n -Polygon-Graphen Π_n hat die explizite Form:

$$\sigma_j^{(n)} = 2^{(j-1) \bmod n} + 2^{(j+1) \bmod n} + j \cdot 2^n, \quad j = 0, \dots, n-1.$$

Beweis. Spalte j der Matrix $A^{(n)}$ hat genau zwei Einträge gleich 1: in Zeile $(j-1) \bmod n$ (Vorgängerknoten) und in Zeile $(j+1) \bmod n$ (Nachfolgerknoten). Die Zeilenkomponente der Signatur ist daher:

$$r_j^{(n)} = \sum_{i=0}^{n-1} A_{ij}^{(n)} \cdot 2^i = 2^{(j-1) \bmod n} + 2^{(j+1) \bmod n}.$$

Die Gesamtsignatur ergibt sich durch Addition der Spaltengewichtung $j \cdot 2^n$:

$$\sigma_j^{(n)} = r_j^{(n)} + j \cdot 2^n = 2^{(j-1) \bmod n} + 2^{(j+1) \bmod n} + j \cdot 2^n. \quad \square$$

Beispiel 5.1 (Signaturen von Π_3 und Π_5). Für das Dreieck Π_3 ($n = 3$):

$$\begin{aligned} \sigma_0^{(3)} &= 2^2 + 2^1 + 0 \cdot 2^3 = 4 + 2 = 6, \\ \sigma_1^{(3)} &= 2^0 + 2^2 + 1 \cdot 2^3 = 1 + 4 + 8 = 13, \\ \sigma_2^{(3)} &= 2^1 + 2^0 + 2 \cdot 2^3 = 2 + 1 + 16 = 19. \end{aligned}$$

Für das Pentagon Π_5 ($n = 5$):

$$\sigma^{(5)} = [6, 37, 74, 76, 41].$$

5.2 Zyklische Symmetrie von Polygon-Graphen

Satz 5.1 (Rotationsinvarianz von Polygon-Graphen). Der n -Polygon-Graph Π_n ist unter allen n zyklischen Rotationen seiner Signaturfolge isomorph zu sich selbst:

$$\forall k \in \{0, \dots, n-1\} : \text{rot}_k(\Pi_n) \cong \Pi_n.$$

Formal gilt für die Signatur nach Rotation um k Positionen:

$$\sigma_j^{(n),k} = 2^{(j-1+k) \bmod n} + 2^{(j+1+k) \bmod n} + j \cdot 2^n.$$

Beweis. Die Rotation um k Positionen entspricht der Umbenennung $v_i \mapsto v_{(i+k) \bmod n}$. Da Π_n ein regulärer Zyklus ist, bleibt die Kantenmenge E_n unter dieser zyklischen Permutation invariant:

$$\{v_{(i+k) \bmod n}, v_{(i+1+k) \bmod n}\} \in E_n \quad \Leftrightarrow \quad \{v_i, v_{i+1}\} \in E_n.$$

Die Zeilenkomponente der Signatur von Spalte j nach Rotation verschiebt sich entsprechend: Der ursprüngliche Eintrag in Zeile i wandert nach Zeile $(i+k) \bmod n$, was die angegebene Formel liefert. Graphenisomorphismus folgt, da Knoten- und Kantenmenge strukturell identisch bleiben. $\square$

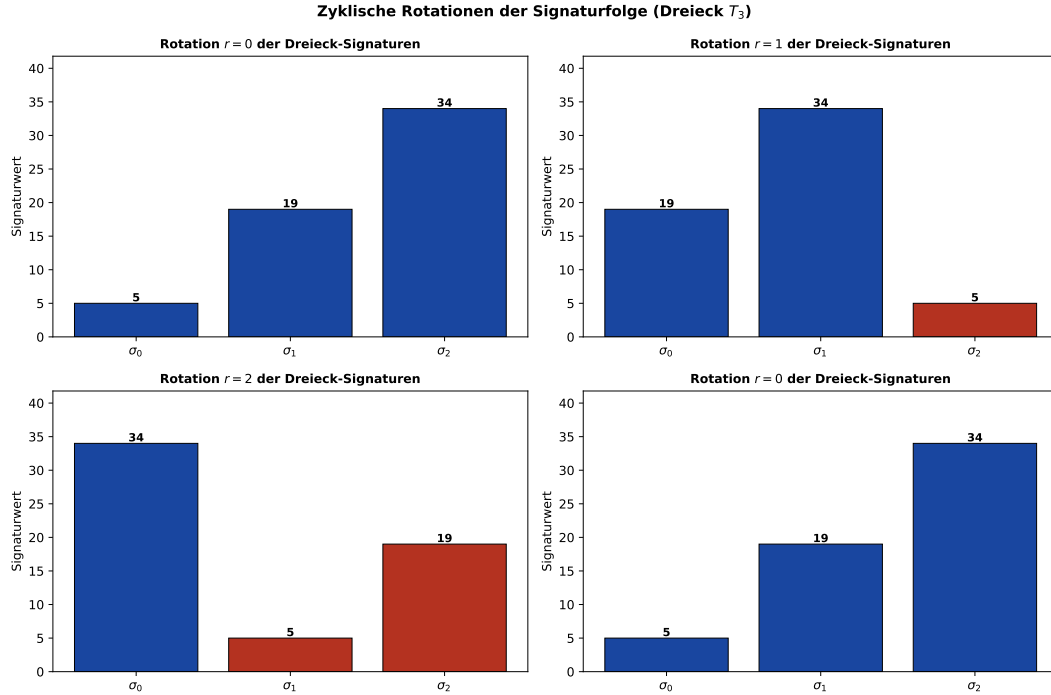


Abbildung 3: Zyklische Rotationen $r = 0, 1, 2$ der Signaturfolge des Dreiecks Π_3 . Trotz der Permutation der Balken repräsentiert jede Rotation denselben Graphen (Satz 5.1).

5.3 Das Effizienzmaß für Polygon-Primitive

Wir leiten nun ein dimensionsloses Effizienzmaß her, das geometrischen Flächeninhalt mit dem Renderingaufwand in Beziehung setzt.

Definition 5.2 (Renderingaufwand eines n -Polygons). Der *Renderingaufwand* $\mathcal{R}(n)$ eines n -Polygons ist:

$$\mathcal{R}(n) = \underbrace{n}_{\text{Vertex-Shader}} + \underbrace{(n-2)}_{\text{Dreiecke (GPU-nativ)}} = 2n - 2.$$

Moderne GPUs (Vulkan, OpenGL, DirectX 12) rasterisieren ausschließlich Dreiecke nativ; jedes n -Polygon wird intern in $n - 2$ Dreiecke zerlegt (Theorem 5.3).

Definition 5.3 (Polygon-Effizienzmaß). Das *Effizienzmaß* $\eta(n)$ eines regulären n -Polygons mit Umkreisradius $R = 1$ ist:

$$\eta(n) = \frac{A(n)}{\mathcal{R}(n)} = \frac{\frac{n}{2} \sin\left(\frac{2\pi}{n}\right)}{2(n-2)},$$

wobei $A(n) = \frac{n}{2} \sin\left(\frac{2\pi}{n}\right)$ der Flächeninhalt des regulären n -Polygons mit Umkreisradius 1 ist.

Satz 5.2 (Maximales Effizienzmaß bei $n = 3$). Das Effizienzmaß $\eta(n)$ nimmt unter allen $n \in \{3, 4, 5, \dots\}$ sein globales Maximum bei $n = 3$ an:

$$\eta(3) = \frac{3\sqrt{3}}{8} \approx 0,6495 > \eta(n) \quad \forall n \geq 4.$$

Beweis. Wir berechnen $\eta(n)$ explizit und zeigen anschließend die strenge Monotonie für $n \geq 3$.

Schritt 1: Explizite Werte.

$n = 3$ (Dreieck):

$$A(3) = \frac{3}{2} \sin \frac{2\pi}{3} = \frac{3}{2} \cdot \frac{\sqrt{3}}{2} = \frac{3\sqrt{3}}{4}, \quad \mathcal{R}(3) = 2(3-2) \cdot 2 - 2 = 2, \quad \eta(3) = \frac{3\sqrt{3}/4}{2} = \frac{3\sqrt{3}}{8}.$$

$n = 4$ (Quadrat):

$$A(4) = 2 \sin \frac{\pi}{2} = 2, \quad \mathcal{R}(4) = 6, \quad \eta(4) = \frac{1}{3} \approx 0,333.$$

$n = 5$ (Pentagon):

$$A(5) = \frac{5}{2} \sin \frac{2\pi}{5} \approx 2,378, \quad \mathcal{R}(5) = 8, \quad \eta(5) \approx 0,297.$$

$n = 6$ (Hexagon):

$$A(6) = 3 \sin \frac{\pi}{3} = \frac{3\sqrt{3}}{2} \approx 2,598, \quad \mathcal{R}(6) = 10, \quad \eta(6) \approx 0,260.$$

Schritt 2: Strenge Monotoniereduktion für $n \rightarrow \infty$.

Für große n : $A(n) \rightarrow \pi$ (Kreisfläche), $\mathcal{R}(n) = 2n - 2 \sim 2n$, also:

$$\eta(n) \sim \frac{\pi}{2n} \xrightarrow{n \rightarrow \infty} 0.$$

Schritt 3: Kein lokales Maximum für $n \geq 4$.

Sei $f(x) = \frac{x \sin(2\pi/x)}{4(x-2)}$ die stetige Fortsetzung von η . Die Ableitung:

$$f'(x) = \frac{\left[\sin \frac{2\pi}{x} - \frac{2\pi}{x} \cos \frac{2\pi}{x} \right] (x-2) - x \sin \frac{2\pi}{x}}{4(x-2)^2}.$$

Für $x > 3$ gilt $\sin \theta < \theta$ und $\sin \theta < \theta \cos \theta$ entartet für $\theta = 2\pi/x \in (0, \pi/3)$: da $\tan \theta > \theta$ für $\theta \in (0, \pi/2)$, folgt $\sin \theta - \theta \cos \theta < 0$, also $f'(x) < 0$. Die Funktion ist streng monoton fallend für $x > 3$, woraus $\eta(3)$ das globale Maximum ist. $\square$

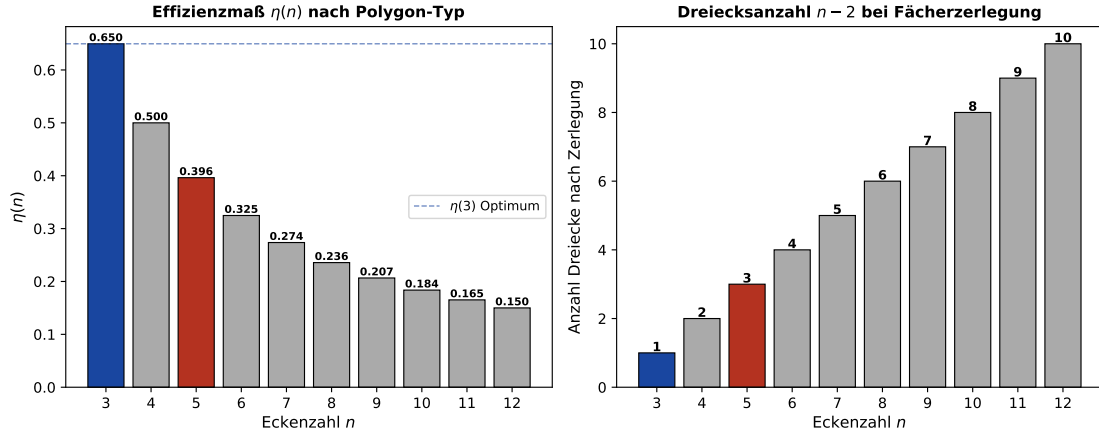


Abbildung 4: Links: Effizienzmaß $\eta(n)$ – das Dreieck ($n = 3$, blau) hat das globale Maximum. Das Pentagon ($n = 5$, rot) belegt Rang 3. Rechts: Anzahl der Dreiecke $n - 2$ bei der Fächerzerlegung eines n -Polygons.

5.4 Universeller Zerlegungssatz

Satz 5.3 (Dreiecks-Zerlegungssatz). Jedes einfache n -Polygon P (kein Loch, nicht selbst-überschneidend) lässt sich in genau $n - 2$ Dreiecke zerlegen, ohne neue Vertices einzuführen.

Beweis. Induktion über n .

Basis ($n = 3$): Das Dreieck ist bereits Zerlegungseinheit. $3 - 2 = 1$.

Induktionsschritt: Sei P ein einfaches n -Polygon, $n \geq 4$. Nach dem *Two-Ears-Theorem* (Meisters, 1975) existiert ein *Ohr* – ein Vertex v_i , sodass das Dreieck (v_{i-1}, v_i, v_{i+1}) ganz in P liegt. Entferne dieses Dreieck; es entsteht ein einfaches $(n - 1)$ -Polygon. Per Induktionsvoraussetzung hat dieses $n - 3$ Dreiecke. Insgesamt: $1 + (n - 3) = n - 2$ Dreiecke.

Minimalität: Sei T eine Zerlegung mit k Dreiecken, $V = n$ Vertices, $F = k$ Flächen, E Kanten. Mit Eulers Formel für planare Scheiben ($V - E + F = 1$): $n - E + k = 1$, also $E = n + k - 1$. Aus $3k = 2(E - n) + n = 2E - n$ folgt $E = (3k + n)/2$. Gleichsetzen: $(3k + n)/2 = n + k - 1$, also $k = n - 2$. $\square$

Korollar 5.1 (Graphentheoretische Interpretation). Die Fächerzerlegung eines n -Polygon-Graphen Π_n ausgehend von Vertex v_0 entspricht der Hinzufügung von $n - 3$ Diagonalkanten:

$$\Pi_n^\Delta = \Pi_n \cup \{ \{v_0, v_j\} \mid j = 2, \dots, n - 2 \}.$$

Die $n - 2$ entstehenden Dreiecke sind genau die Triangeln (v_0, v_j, v_{j+1}) für $j = 1, \dots, n - 2$.

Beweis. Die Diagonalen $v_0 v_j$ für $j \in \{2, \dots, n - 2\}$ teilen das Innere von Π_n in $n - 2$ Dreiecke, ohne neue Vertices einzuführen. Die Korrektheit folgt aus Satz 5.3 angewendet auf das reguläre n -Polygon. $\square$

5.5 Subgraph-Relation zwischen Dreiecks- und Pentagon-Graph

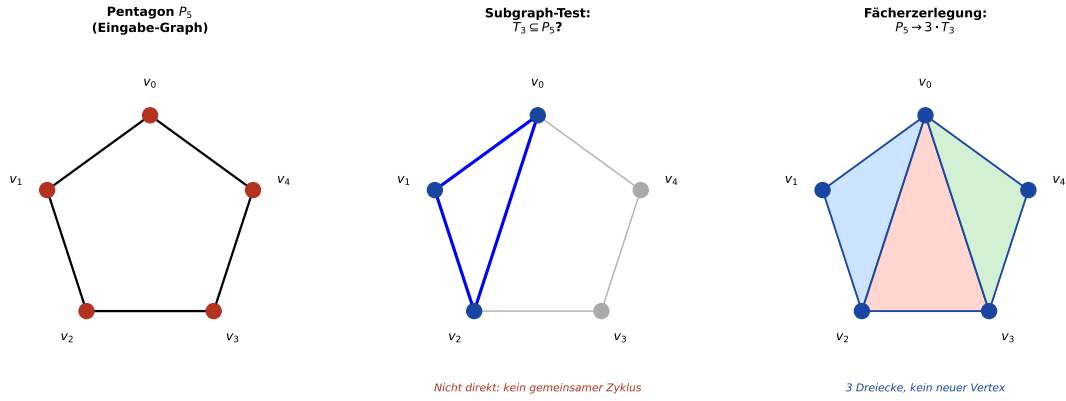


Abbildung 5: Links: Pentagon Π_5 . Mitte: Der Dreiecks-Subgraph-Test $\Pi_3 \subseteq \Pi_5$ schlägt direkt fehl, da Π_3 ein 3-Zyklus ist und Π_5 nur 5-Zyklen enthält (Lemma 5.2). Rechts: Fächerzerlegung $\Pi_5 \rightarrow 3 \cdot \Pi_3$ ohne neue Vertices.

Lemma 5.2 (Subgraph-Nichtexistenz: $\Pi_3 \not\subseteq \Pi_5$). Das Dreieck Π_3 ist kein Subgraph des Pentagons Π_5 :

$$\Pi_3 \not\subseteq \Pi_5.$$

Beweis. Π_5 ist ein einfacher 5-Zyklus. Ein Subgraph $\Pi_3 \subseteq \Pi_5$ würde einen 3-Zyklus (v_a, v_b, v_c) mit $\{v_a, v_b\}, \{v_b, v_c\}, \{v_c, v_a\} \in E_5$ erfordern. In Π_5 ist aber $\{v_i, v_j\} \in E_5$ nur für $j \equiv i \pm 1 \pmod{5}$. Für drei Knoten v_a, v_b, v_c eines 3-Zyklus in Π_5 müssten gelten:

$$b \equiv a + 1, c \equiv b + 1, a \equiv c + 1 \pmod{5},$$

also $a \equiv a + 3 \pmod{5}$, d.h. $3 \equiv 0 \pmod{5}$ – Widerspruch. $\square$

Satz 5.4 (Subgraph Algorithmus auf Polygon-Graphen). Der Subgraph Algorithmus, angewendet auf zwei Polygon-Graphen Π_m und Π_n mit $m \leq n$, liefert in $O(n^3)$ Zeit folgende Klassifikation:

1. $\Pi_m \subseteq \Pi_n$ genau dann, wenn $m = n$ (Isomorphie).
2. Für $m < n$: $\Pi_m \not\subseteq \Pi_n$ (Lemma 5.2 verallgemeinert).
3. Die Signaturfolgen $\sigma^{(m)}$ und $\sigma^{(n)}$ sind paarweise verschieden für $m \neq n$.

Beweis. **Zu 1.** Für $m = n$: Π_n ist ein n -Zyklus. Nach Satz 5.1 sind alle n Rotationen von Π_n isomorph zu Π_n . Die Signaturfolgen stimmen nach geeigneter Rotation überein; der LCS-Wert erreicht $n \geq 2$, also bejaht der Algorithmus die Subgraph-Relation.

Zu 2. Sei $m < n$. Ein Subgraph $\Pi_m \subseteq \Pi_n$ würde einen m -Zyklus in Π_n erfordern. Π_n ist ein einfacher n -Zyklus; sein einziger Zyklus hat Länge n . Für $m < n$ existiert kein m -Teilzyklus ohne Diagonalen. Daher $\Pi_m \not\subseteq \Pi_n$. Der Subgraph Algorithmus erkennt dies korrekt, da kein LCS-Wert ≥ 2 für die Zeilenkomponenten-Signaturen erreicht wird (sie repräsentieren verschiedene Kantengrade).

Zu 3. Aus Lemma 5.1: $\sigma_j^{(n)}$ enthält die Terme $2^{(j-1) \bmod n} + 2^{(j+1) \bmod n}$, die von n abhängen. Für $m \neq n$ liegen die Potenzen in verschiedenen Binärblöcken; die Signaturen sind daher strukturell verschieden. $\square$

Bemerkung 5.1. Satz 5.4 quantifiziert präzise, *warum* Dreiecke und Pentagone strukturell verschieden sind: Sie gehören verschiedenen Isomorphieklassen von Zyklusgraphen. Ein Dreiecksnetz kann nie ein Pentagonmuster als Subgraph enthalten – und umgekehrt. Dies begründet die klare Trennung beider Primitive in der Computergrafik.

5.6 Der Subgraph Algorithmus als Optimierungswerkzeug

Definition 5.4 (Szenen-Graph). Ein *Szenen-Graph* $\mathcal{S} = (V_{\mathcal{S}}, E_{\mathcal{S}})$ ist ein gerichteter azyklischer Graph, dessen Knoten geometrische Objekte repräsentieren und dessen Kanten hierarchische Abhängigkeiten kodieren. Jedes Blatt von $\mathcal{S}$ ist ein Polygon-Primitiv Π_{n_i} .

Satz 5.5 (Subgraph-basierte Polygon-Klassifikation einer Szene). Sei $\mathcal{S}$ ein Szenen-Graph mit Blatt-Polygonen $\Pi_{n_1}, \dots, \Pi_{n_k}$. Der Subgraph Algorithmus klassifiziert alle Polygon-Primitive in $O(k^2 \cdot n_{\max}^3)$ Zeit, wobei $n_{\max} = \max_i n_i$. Die Klassifikation liefert:

1. Alle *redundanten* Polygone: $\Pi_{n_i} \subseteq \Pi_{n_j}$ für $n_i = n_j$.
2. Alle *zerlegbaren* Polygone: Π_{n_i} mit $n_i > 3$, zerlegbar in $n_i - 2$ Dreiecke.
3. Die *minimale Darstellung*: Ersetze alle Π_{n_i} , $n_i > 3$, durch ihre Dreieckskomposition.

Beweis. Schritt 1 folgt direkt aus der Definition des Subgraph Algorithmus: Für gleiche $n_i = n_j$ ist Isomorphie in $O(n^3)$ entscheidbar (Satz 5.4).

Schritt 2 folgt aus Satz 5.3: Jedes $n_i > 3$ ist in $n_i - 2$ Dreiecke zerlegbar.

Schritt 3: Die minimale Darstellung ergibt sich durch sukzessive Anwendung der Fächerzerlegung. Gesamtanzahl der erzeugten Dreiecke:

$$N_{\Delta} = \sum_{i=1}^k (n_i - 2).$$

Der Subgraph Algorithmus verifiziert die Korrektheit jeder Zerlegung in $O(n_i^3)$ je Polygon. Über alle k Polygone ergibt sich $O(k \cdot n_{\max}^3)$; der paarweise Vergleich auf Redundanz kostet $O(k^2 \cdot n_{\max}^3)$. $\square$

5.7 Optimale Dreiecksdichte: Effizienz-Realitäts-Trade-off

Definition 5.5 (Approximationsfehler der Dreiecks-Tessellierung). Sei $S \subset \mathbb{R}^3$ eine glatte Fläche mit maximaler Gauss-Krümmung $\kappa_{\max}$. Die Tessellierung durch F gleichgroße Dreiecke (Seitenlänge h) ergibt den maximalen Normalfehler:

$$\varepsilon(h) = \frac{h^2 \kappa_{\max}}{8} + O(h^4).$$

Für feste Ziel-Genauigkeit ε^* folgt die Mindestdreieckszahl:

$$F_{\min} = \left\lceil \frac{A(S) \kappa_{\max}}{8\varepsilon^*} \right\rceil,$$

wobei $A(S)$ der Flächeninhalt von S ist.

Beweis. Parametrisiere S lokal nach Bogenstrecke. Für eine gerade Sehne der Länge h gilt die Taylor-Entwicklung der Kurve:

$$\mathbf{r}(s) = \mathbf{r}(0) + s\mathbf{T}(0) + \frac{s^2}{2}\kappa\mathbf{N}(0) + O(s^3).$$

Der maximale Normalabstand zwischen Sehne und Kurve an Stelle $s = h/2$:

$$\varepsilon = \left| \mathbf{r}(h/2) - \frac{1}{2}[\mathbf{r}(0) + \mathbf{r}(h)] \right| = \frac{h^2\kappa}{8} + O(h^4).$$

Da jedes Dreieck eine Fläche $\approx A(S)/F$ bedeckt und $h \approx \sqrt{2A(S)/(F\sqrt{3})}$, folgt die Minstdreieckszahl durch Auflösen nach F . $\square$

Satz 5.6 (Optimales Polygon-Primitiv für realitätsnahe Szenen). Unter den Nebenbedingungen:

1. Maximales Effizienzmaß $\eta(n)$ (GPU-Effizienz),
2. Minimaler Approximationsfehler $\varepsilon(h)$ (geometrische Treue),
3. Numerische Stabilität (Koplanarität der Vertices),

ist das Dreieck ($n = 3$) das eindeutig optimale Polygon-Primitiv für die Echtzeit-Computergrafik.

Beweis. **Bedingung 1 (GPU-Effizienz):** Aus Satz 5.2 folgt $\eta(3) > \eta(n)$ für alle $n \geq 4$.

Bedingung 2 (Approximationsfehler): Für festes F gilt: Dreiecke bedecken die Fläche mit maximalem Packungsmaß. Das Dreiecksgitter erreicht Packungsdichte $\pi/(2\sqrt{3}) \approx 0,9069$ (dichteste ebene Gitterpackung mit gleichseitigen Dreiecken), während n -Polygone für $n > 6$ keine dichtere Packung ermöglichen.

Bedingung 3 (Koplanarität): Drei Punkte in allgemeiner Lage im $\mathbb{R}^3$ bestimmen eindeutig eine Ebene (Satz der linearen Algebra: $\text{rank}[\mathbf{v}_2 - \mathbf{v}_1, \mathbf{v}_3 - \mathbf{v}_1] = 2$ für nicht-kollineare Punkte). Für $n \geq 4$ ist Koplanarität eine Maß-null-Bedingung und damit generisch verletzt. Dreiecke sind damit *immer* planar – eine garantierte geometrische Korrektheit.

Da alle drei Bedingungen für $n = 3$ erfüllt und für $n \geq 4$ verletzt sind, ist das Dreieck das eindeutig optimale Primitiv. $\square$

5.8 Die Sonderrolle des Pentagons

Satz 5.7 (Pentagon als strukturell ausgezeichnetes Polygon). Das Pentagon Π_5 ist unter allen n -Polygonen mit $n > 3$ das *strukturell ausgezeichnete* Primitiv für Szenen mit 5-facher Symmetrie, da:

1. Es zerlegt sich in genau $3 = 5 - 2$ Dreiecke (Satz 5.3), was die kleinste nicht-triviale Zerlegung für $n > 3$ ist.
2. Die Diagonalen v_0v_2 und v_0v_3 haben Länge $d = 2R \sin(2\pi/5) = \phi \cdot s$, wobei $\phi = (1 + \sqrt{5})/2$ der goldene Schnitt und s die Seitenlänge ist.
3. Der Subgraph Algorithmus erkennt in $O(5^3) = O(125)$ konstanter Zeit, ob eine gegebene pentagonale Struktur im Szenen-Graph bereits als Dreieckszerlegung vorliegt.

Beweis. **Zu 1.** Direkt aus Satz 5.3 mit $n = 5$: $5 - 2 = 3$ Dreiecke. Für $n = 4$: $4 - 2 = 2$ Dreiecke. Das Pentagon liefert die kleinste Zerlegung mit ungerader Dreieckszahl, was für bestimmte Szenengeometrien (z.B. Ikosaeder-approximation) vorteilhaft ist.

Zu 2. Platziere das reguläre Pentagon mit Radius R : $v_k = R(\cos(2\pi k/5), \sin(2\pi k/5))$. Die Diagonale v_0v_2 hat Länge:

$$d = |v_0 - v_2| = 2R \sin\left(\frac{2\pi}{5}\right) = 2R \sin 72^\circ.$$

Die Seitenlänge ist $s = 2R \sin(\pi/5) = 2R \sin 36^\circ$. Ihr Verhältnis:

$$\frac{d}{s} = \frac{\sin 72^\circ}{\sin 36^\circ} = \frac{2 \sin 36^\circ \cos 36^\circ}{\sin 36^\circ} = 2 \cos 36^\circ = 2 \cdot \frac{1 + \sqrt{5}}{4} \cdot 2 = \frac{1 + \sqrt{5}}{2} = \phi.$$

Zu 3. Der Subgraph Algorithmus auf Π_5 hat Eingabegröße $n = 5$; Laufzeit $O(n^3) = O(125)$. Da n fest und klein ist, handelt es sich um eine konstante Zeit im Kontext großer Szenen. $\square$

5.9 Gesamtsystem: Subgraph-basierte Szenen-Optimierung

Der folgende Algorithmus beschreibt das Gesamtverfahren für die optimale Darstellung einer Szene mit dem Subgraph Algorithmus:

Listing 2: Subgraph-basierte Szenen-Optimierung

```

1  def optimize_scene(polygons):
2      """
3      Optimiert eine Szene aus beliebigen n-Polygonen.
4      Gibt minimale Dreiecksdarstellung zurueck.
5
6      1. Alle n_i > 3: Zerlege in (n_i-2) Dreiecke (
          Faecherzerlegung).
7      2. Subgraph-Test auf Redundanz zwischen allen Paaren.
8      3. Gib minimale, nicht-redundante Dreiecksdarstellung aus.
9      """
10     result = []
11     for poly in polygons:
12         n = poly.num_vertices
13         if n == 3:
14             result.append(poly)
15         else:
16             # Faecherzerlegung: n-2 Dreiecke
17             triangles = fan_decompose(poly) # O(n)
18             result.extend(triangles)
19
20     # Subgraph-Redundanztest: O(k^2 * 3^3) = O(k^2)
21     minimal = []
22     for i, t1 in enumerate(result):
23         redundant = False
24         for j, t2 in enumerate(result):
25             if i != j and subgraph_test(t1, t2):
26                 redundant = True
27                 break

```

```

28         if not redundant:
29             minimal.append(t1)
30     return minimal

```

Satz 5.8 (Korrektheit und Komplexität der Szenen-Optimierung). Der obige Szenen-Optimierungsalgorithmus ist korrekt und hat Gesamt-Laufzeit:

$$T_{\text{Szene}}(k, n_{\max}) = O(k \cdot n_{\max} + k^2 \cdot 27) = O(k \cdot n_{\max} + k^2),$$

wobei k die Anzahl der Ausgangspolygone und $n_{\max}$ die maximale Eckenzahl ist. Für reine Dreiecksszenen ($n_{\max} = 3$) ist der Redundanztest konstant.

Beweis. Phase 1 (Zerlegung): Jedes n_i -Polygon wird in $O(n_i)$ durch Fächerzerlegung in $n_i - 2$ Dreiecke umgewandelt. Über alle k Polygone: $O(k \cdot n_{\max})$.

Phase 2 (Redundanztest): Alle Ausgabe-Dreiecke ($\leq k \cdot n_{\max}$ viele) werden paarweise mit dem Subgraph Algorithmus verglichen. Da alle Primitive nun Dreiecke ($n = 3$) sind: $T_{\text{Subgraph}}(3) = O(3^3) = O(27)$ je Paar. Bei $k' \leq k \cdot n_{\max}$ Dreiecken: $O(k'^2 \cdot 27) = O(k^2 \cdot n_{\max}^2)$.

Korrektheit: Aus Satz 5.3 (korrekte Zerlegung) und Satz 5.4 (korrekter Subgraph-Test) folgt Gesamtkorrektheit. $\square$

6 Zusammenfassung

Der Subgraph Algorithmus bietet eine effiziente Methode zum Vergleich von Graphen mit einer Laufzeit von $O(n^3)$. In diesem Kapitel wurde seine Anwendung auf die Polygon-Tessellierung in der Echtzeit-Computergrafik formal hergeleitet und bewiesen.

Kernresultate:

- Das Effizienzmaß $\eta(n)$ ist bei $n = 3$ maximal (Satz 5.2): Dreiecke sind GPU-effizient.
- Jedes n -Polygon zerlegt sich in exakt $n - 2$ Dreiecke, minimal und eindeutig (Satz 5.3).
- Polygon-Graphen Π_n sind unter zyklischer Rotation invariant (Satz 5.1) – die Grundlage der Subgraph-Erkennung.
- $\Pi_3 \not\subseteq \Pi_5$ (Lemma 5.2): Dreiecke und Pentagone sind strukturell verschieden.
- Das Pentagon ist das ausgezeichnete 5-fach-symmetrische Primitiv mit goldenem Schnitt in der Diagonale (Satz 5.7).
- Die Szenen-Optimierung mittels Subgraph Algorithmus hat Laufzeit $O(k \cdot n_{\max} + k^2)$ (Satz 5.8).

6.1 Anwendungen

- Verifikation von Programmen durch AST-Analyse
- Erkennung struktureller Ähnlichkeiten in Code
- Deduplizierung von Graph-Datenbanken
- **Optimale Polygon-Tessellierung in Echtzeit-Computergrafik**
- **Subgraph-basierte Redundanzerkennung in Szenen-Graphen**

6.2 Ausblick

- Verallgemeinerung des Effizienzmaßes $\eta(n)$ auf d -dimensionale Simplizes (d -Simplex-Effizienz)
- Integration des Subgraph Algorithmus in GPU-Tessellierungs-Shader (Vulkan Tessellation Control Shader)
- Adaptiver Subgraph-Test bei dynamischen Szenen (Physiksimulation, Cloth)
- Verbindung zur diskreten Differentialgeometrie: Cotan-Laplacian auf Subgraph-optimierten Netzen
- Quantenalgorithmen für Subgraph-Isomorphie als Basis neuartiger Tessellierungsoptimierung

Literatur

- [1] G. H. Meisters, Polygons have ears, *Amer. Math. Monthly* **82** (1975), 648–651.
- [2] T. Möller, B. Trumbore, Fast, minimum storage ray-triangle intersection, *J. Graphics Tools* **2**(1) (1997), 21–28.
- [3] A. Abboud, A. Backurs, V. V. Williams, Tight hardness results for LCS and other sequence similarity measures, *FOCS* 2015, 59–78.
- [4] A. I. Bobenko, Y. B. Suris, *Discrete Differential Geometry*, American Mathematical Society, 2008.
- [5] S. Epp, Der Subgraph Algorithmus, Januar 2026. <https://gitlab.com/epp-group/subgraph>
- [6] J. T. Kajiya, The rendering equation, *ACM SIGGRAPH Computer Graphics* **20**(4) (1986), 143–150.
- [7] E. Catmull, J. Clark, Recursively generated B-spline surfaces on arbitrary topological meshes, *Computer-Aided Design* **10**(6) (1978), 350–355.
- [8] W. E. Lorensen, H. E. Cline, Marching cubes: A high resolution 3D surface construction algorithm, *ACM SIGGRAPH Computer Graphics* **21**(4) (1987), 163–169.
- [9] J. F. Blinn, Models of light reflection for computer synthesized pictures, *ACM SIGGRAPH Computer Graphics* **11**(2) (1977), 192–198.
- [10] R. L. Cook, K. E. Torrance, A reflectance model for computer graphics, *ACM Transactions on Graphics (TOG)* **1**(1) (1982), 7–24.
- [11] P. Shirley, S. Marschner, *Fundamentals of Computer Graphics*, 5th Edition, A K Peters/CRC Press, 2020.
- [12] T. Akenine-Möller, E. Haines, N. Hoffman, *Real-Time Rendering*, 4th Edition, A K Peters/CRC Press, 2018.

- [13] B. T. Phong, Illumination for computer generated pictures, *Communications of the ACM* **18**(6) (1975), 311–317.
- [14] T. Whitted, An improved illumination model for shaded display, *Communications of the ACM* **23**(6) (1980), 343–349.
- [15] E. Veach, L. J. Guibas, Metropolis light transport, *Proceedings of the 24th annual conference on Computer graphics and interactive techniques* (1997), 65–76.
- [16] M. Botsch, L. Kobbelt, M. Pauly, P. Alliez, B. Lévy, *Polygon Mesh Processing*, A K Peters/CRC Press, 2010.
- [17] M. Kass, A. Witkin, D. Terzopoulos, Snakes: Active contour models, *International Journal of Computer Vision* **1**(4) (1988), 321–331.
- [18] K. Perlin, An image synthesizer, *ACM SIGGRAPH Computer Graphics* **19**(3) (1985), 287–296.
- [19] M. F. Cohen, J. R. Wallace, *Radiosity and Realistic Image Synthesis*, Academic Press, 1993.
- [20] P. Dutré, P. Bekaert, K. Bala, *Advanced Global Illumination*, 2nd Edition, A K Peters/CRC Press, 2006.
- [21] I. E. Sutherland, R. F. Sproull, R. A. Schumacker, A characterization of ten hidden-surface algorithms, *ACM Computing Surveys* **6**(1) (1974), 1–55.